Quest #76

Referred

CAREER 180

Hiring Quest – Senior Full-Stack Laravel Engineer @ Career 180

Challenge-based hiring quest with structured evaluation and real project outcomes.

Status: Referred **Registration:** February 27, 2026 **Submission:** February 28, 2026

META INFO

CATEGORY	STATUS
Referred	Referred
REGISTRATION DEADLINE	**SUBMISSION DEADLINE**
February 27, 2026	February 28, 2026

PRIZE

Top performers get hired with a paid contract and the opportunity to work on real-world projects.

QUEST BRIEF

We are **Career 180**, a comprehensive and interactive Career Knowledge Network where all career-related services are centralized in one place.

We're building a scalable Learning Management System (LMS) that powers interactive learning experiences for thousands of learners.

We're hiring a **Senior Full-Stack Laravel Engineer (6+ YOE)** to architect and implement the core foundation of our next-generation LMS.

Start Date: Immediate
Location: Hybrid – Nasr City (3 days onsite)
Salary: EGP 50 – 80K

How the Hiring Quest Works

1. Register
2. Receive detailed instructions
3. Submit before deadline
4. Top candidates attend a deep technical review session
5. One candidate will be hired

Who We're Looking For

1. 5+ years of professional Laravel development experience
2. Strong in **OOP, HTTP, REST/JSON, queues, caching, transactions, validation**
3. Deep understanding of **concurrency, data integrity, race condition prevention**
4. Solid grasp of **Livewire v3** and **Alpine.js** patterns
5. Proficient in automated testing (**Pest** or PHPUnit)
6. Comfortable with **Action/Service classes**, Composer, NPM, Vite, Tailwind, and Git
7. Strong database skills (indexes, constraints, performance awareness)
8. Bonus: experience with **Filament v3**
9. Bonus: event-driven architecture / high-scale systems

Your Mission

Build the **core foundation of a scalable Mini-LMS**.

This is not just a feature exercise — it's a systems-thinking exercise.

Tech Stack

1. Laravel 11/12
2. Livewire v3
3. Alpine.js
4. Filament v3
5. Pest
6. MySQL
7. Plyr.js
8. **Docker + Docker Compose (Required)**

Core Functional Scope (Focused)

Public Home

1. / lists published courses (title, image, level)
2. Must be efficient (no N+1)

Authentication & Enrollment

1. Registration sends **Welcome Email** (queued/async)
2. Enrollment requires authentication
3. Enrollment must not create duplicates
4. Enrollment must not allow draft courses
5. System must behave correctly under:
    1. rapid clicks
    2. network retries
    3. queue retries

Course View

/courses/{slug}

1. Course details
2. Ordered lessons
3. Free preview lessons accessible to guests
4. Unique slug handling (including soft deletes)

🌊 Lesson & Progress

/courses/{slug}/lessons/{lesson}

1. Video player via Plyr
2. Mark as completed
3. Track started_at, completed_at

Completion rules:

1. A course becomes completed when all required lessons are completed
2. On completion:
    1. Create certificate record (UUID)
    2. Send completion email (async, once only)

Courses may be edited after users enroll (lessons added/removed).
System must remain logically consistent.
(No further clarification will be provided.)

Required Alpine Behaviors (Min 3)

1. Accordion lesson list
2. Confirmation modal before marking completed
3. Animated progress bar
4. Plyr lifecycle integration
5. (Bonus) Dark mode toggle

Architectural Requirements

1. Core flows implemented via **Action classes**
2. Use **Events + queued listeners** where appropriate
3. Use **DB constraints** where appropriate
4. Use **transactions** in multi-step flows
5. Policies must isolate user data and prevent cross-user exposure

---

Docker Requirements (Mandatory)

Your submission must include a working Dockerized setup with:

Required services

1. app (PHP-FPM or Laravel container)
2. web (Nginx or Apache)
3. db (MySQL)
4. queue worker (separate container OR same image different command)
5. *(Optional bonus)* redis for queue/cache

Must-have DX

1. docker-compose.yml at repo root
2. One-command start:
    1. docker compose up --build
3. Documented bootstrap script/commands in README:
    1. install deps
    2. migrations + seeds
    3. queue worker
    4. run tests

The reviewer should be able to run the full project on a clean machine using Docker only.

Testing (Pest Required)

Must cover:

Must cover:

1. Enrollment integrity (idempotency / duplication prevention)
2. Completion flow integrity (email once + certificate once)
3. Slug uniqueness rule
4. User isolation (policy/access)
5. At least one meaningful edge case of your choice

Product Thinking Layer

***Note** : Please focus on explaining your own thinking and decision-making process. We want to understand how you approach product problems and evaluate trade-offs — this is about your thinking, not AI-generated answers.*

In addition to implementation, you must include a section in /docs/PRODUCT_THINKING.md.

In this document, answer:

🌊 Business Risks

What are the top 3 product risks in this LMS?

Explain how your architecture supports or mitigates these risks.

🌊 Metrics That Matter

If you were working with Product, what 5 metrics would you track?

Explain:

1. Where would you capture these metrics?
2. Would you compute or store them?
3. How would you avoid performance issues?

(No need to fully implement analytics — just design thinking.)

🌊 Future Evolution

Imagine in 6 months Career 180 wants:

1. Paid courses

1. Paid courses
2. Mobile app API
3. Corporate multi-tenant accounts
4. Gamification badges

Explain:

1. What in your current architecture supports this?
2. What would need refactoring?

🌊 Trade-offs You Made

List 3 conscious trade-offs you made. And Explain why.

Deliverables

Repo must include:

1. Codebase
2. Migrations + seeders
3. Actions
4. Events + listeners
5. Policies
6. ERD (image or docs)
7. README (setup + assumptions)
8. /docs/ARCHITECTURE.md
9. /docs/PRODUCT_THINKING.md.

10–15 min video:

1. Architecture + trade-offs
2. Concurrency strategy
3. Docker/DX walkthrough
4. What you would improve next

Evaluation Criteria

1. System Architecture & Structure — 30%
2. Data Integrity & Concurrency Handling — 20%
3. Laravel Implementation Quality — 20%
4. Testing Depth & Edge Cases — 15%
5. Admin & UX Implementation — 10%
6. Clarity of Decisions (Video + Docs) — 5%

After Submission

Top candidates will join a **technical review session**

Final decision within **5 business days** after the review.

QUEST SNAPSHOT

STATUS
Referred

REGISTRATION
February 27, 2026

SUBMISSION
February 28, 2026

Quest #76